IPSSI MONTPELLIER · MASTER 2 DATA ENGINEERING · FORMATION APACHE AIRFLOW

TP Jour 3 — Pipeline Immobilier DVF : HDFS + PostgreSQL

TP Jour 3 — Pipeline Immobilier DVF : HDFS + PostgreSQL avec Airflow

Champ	Détail
Module	Apache Airflow — Orchestration de pipelines de données
Niveau	Intermédiaire - Avancé
Durée	3h30
Technos	Apache Airflow 2.8 · HDFS (WebHDFS) · PostgreSQL · Docker Compose
Open Data	DVF — Demandes de Valeurs Foncières (data.gouv.fr)
Rendu	Fichier `dag_dvf.py` + `docker-compose.yaml` + capture de l'UI Airflow

Contexte métier

Vous rejoignez l'équipe Data Engineering d'une startup proptech positionnée sur l'analyse des prix de l'immobilier en France, à l'image de **SeLoger** ou **MeilleursAgents**. Votre mission est de construire un pipeline ETL automatisé qui alimente chaque jour un tableau de bord de suivi des prix au m² par arrondissement parisien.

La source de données est officielle et publique : les **Demandes de Valeurs Foncières (DVF)**, publiées par la Direction Générale des Finances Publiques (DGFiP) sur data.gouv.fr. Ces fichiers recensent

l’ensemble des transactions immobilières notariées en France.

Architecture cible : Data Lake + Data Warehouse

L’architecture repose sur deux zones complémentaires :

- **Zone Raw (Data Lake — HDFS)** : les fichiers CSV bruts sont stockés tels quels dans HDFS. Cette zone constitue la source de vérité immuable. Aucune donnée n’est modifiée à ce stade.
- **Zone Curated (Data Warehouse — PostgreSQL)** : après traitement et agrégation, les résultats enrichis sont insérés dans PostgreSQL. C’est cette base que consomment les dashboards et les équipes métier.

Valeur métier

Le pipeline permet de : - Suivre **quotidiennement** l’évolution du prix médian au m² par arrondissement de Paris - Détecter les **tendances de marché** (hausse, stabilisation, baisse) - Alimenter une **API REST** consommée par l’application mobile (hors périmètre de ce TP) - Conserver un **historique brut** des données sources pour l’audit et la reproductibilité

Architecture du pipeline

Schéma d'architecture

```
data.gouv.fr DVF API
    (CSV open data)
          |
          |  HTTP GET (téléchargement CSV)
          ▼
     ┌──────────────────────┐    WebHDFS REST API    ┌──────────────────────────┐
     │      Airflow         │ ──────────────────────► │      HDFS Cluster        │
     │  (Scheduler +        │  PUT /webhdfs/v1/...    │   NameNode  :9870        │
     │   WebServer)         │ ◄────────────────────── │   DataNode               │
     │                      │  GET /webhdfs/v1/...    │                          │
     └──────────────────────┘                         └──────────────────────────┘
          |                                                        |
          |  Lecture raw CSV depuis HDFS                           |
          |  Filtrage + Agrégation (pandas)                        | stockage
          |                                                        | persistant
          ▼                                                        |
     ┌──────────────────────┐ ◄──────────────────────────────────┘
     │      PostgreSQL      │
     │  (zone curated)      │
     │  prix_m2_arrdt       │
     │  stats_marche        │
     └──────────────────────┘
          |
          |  SQL report (ranking arrondissements)
          ▼
       Logs Airflow / Dashboard
```

Flux de données détaillé

Schéma d'architecture

```
verifier_sources()
          |
          ▼
    telecharger_dvf()          ← HTTP GET data.gouv.fr → dvf_2023.csv (temp)
          |
          ▼
    stocker_hdfs_raw()         ← WebHDFS PUT → /data/dvf/raw/dvf_2023.csv
          |
          ▼
    traiter_donnees()          ← WebHDFS GET → filtre Paris + appartements → agrège
          |
          ▼
    inserer_postgresql()       ← PostgresHook → UPSERT prix_m2_arrondissement
          |
          ▼
    generer_rapport()          ← PostgresHook → SELECT TOP 10 arrondissements
```

Prérequis

Outils à installer

- **Docker Desktop** (version 4.x ou supérieure) — https://www.docker.com/products/docker-desktop
- **Docker Compose v2** (inclus dans Docker Desktop)
- **Python 3.11** avec les packages suivants :

```
apache-airflow==2.8.1
pandas==2.1.4
requests==2.31.0
psycopg2-binary==2.9.9
```

- Un client PostgreSQL (optionnel) : `psql`, DBeaver ou pgAdmin

Connaissances attendues

- Bases d'Apache Airflow (DAGs, Operators, XComs)
- Notions de REST API (méthodes HTTP GET/PUT)
- SQL de base (CREATE TABLE, INSERT, SELECT avec agrégats)

- Concepts Data Lake / Data Warehouse

Vérification de l'environnement

Schéma d'architecture

```
docker --version          # Docker version 24.x.x ou supérieure
    docker compose version    # Docker Compose version v2.x.x
    python --version          # Python 3.11.x
```

Partie 1 — Infrastructure Docker Compose

Objectif

Mettre en place l'ensemble de la stack : Airflow, HDFS (NameNode + DataNode) et PostgreSQL via un unique fichier `docker-compose.yaml`.

Structure des services

Service	Image	Rôle	Port exposé
`postgres`	`postgres:15`	BDD Airflow metadata + BDD DVF curated	5432
`airflow-webserver`	`apache/airflow:2.8.1`	Interface web Airflow	8080
`airflow-scheduler`	`apache/airflow:2.8.1`	Ordonnanceur des DAGs	—
`hdfs-namenode`	`bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8`	Métadonnées HDFS, WebHDFS API	9870, 9000
`hdfs-datanode`	`bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8`	Stockage physique des blocs HDFS	9864

Fichier `docker-compose.yaml`

Créez un fichier `docker-compose.yaml` à la racine de votre projet avec le contenu suivant :

📐 **Schéma d'architecture**

```yaml
version: '3.8'

    x-airflow-common: &airflow-common
      image: apache/airflow:2.8.1
      environment:
        AIRFLOW__CORE__EXECUTOR: LocalExecutor
        AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@pos
        AIRFLOW__CORE__FERNET_KEY: ''
        AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
        AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
        AIRFLOW__WEBSERVER__EXPOSE_CONFIG: 'true'
        _PIP_ADDITIONAL_REQUIREMENTS: "pandas requests psycopg2-binary"
      volumes:
        - ./dags:/opt/airflow/dags
        - ./logs:/opt/airflow/logs
        - ./plugins:/opt/airflow/plugins
      depends_on:
        postgres:
          condition: service_healthy
      networks:
        - dvf-network

    services:

      # ── Base de données ──────────────────────────────────────────────
      postgres:
        image: postgres:15
        container_name: dvf-postgres
        environment:
          POSTGRES_USER: airflow
          POSTGRES_PASSWORD: airflow
          POSTGRES_DB: airflow
        ports:
          - "5432:5432"
        volumes:
          - postgres-data:/var/lib/postgresql/data
          - ./sql/init_dvf.sql:/docker-entrypoint-initdb.d/init_dvf.sql
        healthcheck:
          test: ["CMD", "pg_isready", "-U", "airflow"]
          interval: 10s
          retries: 5
          start_period: 5s
        networks:
          - dvf-network

      # ── Airflow ──────────────────────────────────────────────────────
      airflow-webserver:
        <<: *airflow-common
```

```yaml
    container_name: dvf-airflow-webserver
    command: webserver
    ports:
      - "8080:8080"
    healthcheck:
      test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 30s

  airflow-scheduler:
    <<: *airflow-common
    container_name: dvf-airflow-scheduler
    command: scheduler
    healthcheck:
      test: ["CMD-SHELL", 'airflow jobs check --job-type SchedulerJob --hostname "
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 30s

  airflow-init:
    <<: *airflow-common
    container_name: dvf-airflow-init
    entrypoint: /bin/bash
    command:
      - -c
      - |
        airflow db migrate
        airflow users create \
          --username admin \
          --firstname Admin \
          --lastname User \
          --role Admin \
          --email admin@example.com \
          --password admin
        airflow connections add 'dvf_postgres' \
          --conn-type 'postgres' \
          --conn-host 'postgres' \
          --conn-login 'airflow' \
          --conn-password 'airflow' \
          --conn-port '5432' \
          --conn-schema 'dvf'
    restart: "no"

  # ── HDFS ──────────────────────────────────────────────────────────────
  hdfs-namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: dvf-namenode
    ports:
```

```yaml
      - "9870:9870"   # WebHDFS / UI NameNode
      - "9000:9000"   # HDFS IPC
    environment:
      - CLUSTER_NAME=dvf-cluster
      - CORE_CONF_fs_defaultFS=hdfs://hdfs-namenode:9000
      - CORE_CONF_hadoop_http_staticuser_user=root
      - HDFS_CONF_dfs_webhdfs_enabled=true
      - HDFS_CONF_dfs_permissions_enabled=false
      - HDFS_CONF_dfs_namenode_datanode_registration_ip___hostname___check=false
    volumes:
      - namenode-data:/hadoop/dfs/name
    networks:
      - dvf-network

  hdfs-datanode:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: dvf-datanode
    ports:
      - "9864:9864"   # DataNode UI
    environment:
      - CORE_CONF_fs_defaultFS=hdfs://hdfs-namenode:9000
      - CORE_CONF_hadoop_http_staticuser_user=root
      - HDFS_CONF_dfs_webhdfs_enabled=true
      - HDFS_CONF_dfs_permissions_enabled=false
    volumes:
      - datanode-data:/hadoop/dfs/data
    depends_on:
      - hdfs-namenode
    networks:
      - dvf-network

# ── Volumes & Networks ──────────────────────────────────────────
volumes:
  postgres-data:
  namenode-data:
  datanode-data:

networks:
  dvf-network:
    driver: bridge
```

Démarrage de la stack

📐 **Schéma d'architecture**

```
# Créer les répertoires nécessaires
    mkdir -p dags logs plugins sql

    # Initialiser la base Airflow et le compte admin
    docker compose up airflow-init

    # Démarrer tous les services en arrière-plan
    docker compose up -d

    # Vérifier que tous les services sont sains
    docker compose ps
```

⚠️ **Attention**

Attention : Le démarrage complet peut prendre 2 à 3 minutes. Attendez que tous les services soient en état `healthy` avant de continuer.

Accès aux interfaces

Interface	URL	Identifiants
Airflow Web UI	http://localhost:8080	admin / admin
HDFS NameNode UI	http://localhost:9870	—
DataNode UI	http://localhost:9864	—
PostgreSQL	localhost:5432	airflow / airflow

Partie 2 — API WebHDFS

Présentation

WebHDFS est l'interface REST d'HDFS. Elle permet d'interagir avec le système de fichiers distribué via des requêtes HTTP standard, sans client Hadoop installé. La spécification complète est disponible dans la documentation Apache Hadoop WebHDFS.

L'URL de base dans notre stack est : `http://hdfs-namenode:9870/webhdfs/v1`

Depuis votre machine locale (hors Docker), utilisez : `http://localhost:9870/webhdfs/v1`

Opérations principales

CRÉER UN RÉPERTOIRE

```
PUT http://localhost:9870/webhdfs/v1/data/dvf/raw/?op=MKDIRS&user.name=root
```

Réponse attendue (HTTP 200) :

```
{ } JSON

{"boolean": true}
```

UPLOADER UN FICHIER (EN DEUX ÉTAPES)

WebHDFS utilise une redirection pour l'upload :

Etape 1 — Initier l'upload (retourne un 307 Redirect) :

```
PUT http://localhost:9870/webhdfs/v1/data/dvf/raw/dvf_2023.csv?op=CREATE&user.name=root
```

Etape 2 — Suivre la redirection et envoyer les données (vers le DataNode) :

```
PUT http://<datanode>:9864/webhdfs/v1/data/dvf/raw/dvf_2023.csv?op=CREATE&...
Content-Type: application/octet-stream
Body: <contenu du fichier CSV>
```

Avec la bibliothèque `requests` de Python, passez `allow_redirects=False` à l'étape 1, puis envoyez les données vers l'URL de redirection.

LIRE UN FICHIER

```
GET http://localhost:9870/webhdfs/v1/data/dvf/raw/dvf_2023.csv?op=OPEN&user.name=root
```

LISTER UN RÉPERTOIRE

```
GET http://localhost:9870/webhdfs/v1/data/dvf/raw/?op=LISTSTATUS&user.name=root
```

Réponse (extrait) :

```
{ } JSON

{
      "FileStatuses": {
        "FileStatus": [
          {
            "pathSuffix": "dvf_2023.csv",
            "type": "FILE",
            "length": 134217728,
            "modificationTime": 1704067200000
          }
        ]
      }
    }
```

VÉRIFIER L'EXISTENCE D'UN FICHIER

```
GET http://localhost:9870/webhdfs/v1/data/dvf/raw/dvf_2023.csv?op=GETFILESTATUS&user.na
```

Retourne HTTP 200 si le fichier existe, HTTP 404 sinon.

Classe helper WebHDFS

Créez le fichier `dags/helpers/webhdfs_client.py` avec la structure suivante :

Schéma d'architecture

```
"""
    Client WebHDFS pour interagir avec le cluster HDFS via l'API REST.
    Documentation : https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-h
    """

    import requests
    import logging
    from typing import Optional

    logger = logging.getLogger(__name__)

    WEBHDFS_BASE_URL = "http://hdfs-namenode:9870/webhdfs/v1"
    WEBHDFS_USER = "root"


    class WebHDFSClient:
        """Client léger pour l'API WebHDFS d'Apache Hadoop."""

        def __init__(self, base_url: str = WEBHDFS_BASE_URL, user: str = WEBHDFS_USER)
            self.base_url = base_url
            self.user = user

        def _url(self, path: str, op: str, **params) -> str:
            """Construit l'URL WebHDFS pour une opération donnée."""
            # TODO : construire l'URL complète avec les paramètres op et user.name
            pass

        def mkdirs(self, hdfs_path: str) -> bool:
            """
            Crée un répertoire (et ses parents) dans HDFS.
            Retourne True si succès, lève une exception sinon.
            """
            # TODO : envoyer une requête PUT avec op=MKDIRS
            # Vérifier le code HTTP et parser la réponse JSON
            pass

        def upload(self, hdfs_path: str, local_file_path: str) -> str:
            """
            Uploade un fichier local vers HDFS.
            Retourne le chemin HDFS du fichier uploadé.

            Rappel : WebHDFS upload = 2 étapes
              1. PUT sur le NameNode (allow_redirects=False) -> récupère l'URL de redi
              2. PUT sur le DataNode avec le contenu binaire du fichier
            """
            # TODO : implémenter le double PUT (NameNode -> redirection -> DataNode)
            pass
```

```
    def open(self, hdfs_path: str) -> bytes:
        """
        Lit le contenu d'un fichier HDFS.
        Retourne les données brutes (bytes).
        """
        # TODO : envoyer une requête GET avec op=OPEN
        # Gérer la redirection vers le DataNode
        pass

    def exists(self, hdfs_path: str) -> bool:
        """Vérifie si un fichier ou répertoire existe dans HDFS."""
        # TODO : utiliser op=GETFILESTATUS et inspecter le code HTTP
        pass

    def list_status(self, hdfs_path: str) -> list:
        """Liste le contenu d'un répertoire HDFS."""
        # TODO : utiliser op=LISTSTATUS et parser le JSON
        pass
```

Partie 3 — Schéma PostgreSQL

Objectif

Créer la base de données `dvf` et les tables nécessaires dans PostgreSQL. Ces tables constituent la **zone curated** du pipeline.

Script d'initialisation

Créez le fichier `sql/init_dvf.sql` :

Schéma d'architecture

```sql
-- Création de la base de données dédiée aux données DVF
    CREATE DATABASE dvf;

    \c dvf;

    -- ----------------------------------------------------------------
    -- Table 1 : Transactions brutes Paris (chargées depuis HDFS)
    -- ----------------------------------------------------------------
    CREATE TABLE IF NOT EXISTS dvf_raw (
        id                          SERIAL PRIMARY KEY,
        date_mutation               DATE,
        nature_mutation             VARCHAR(50),
        valeur_fonciere             NUMERIC(15, 2),
        adresse_numero              VARCHAR(10),
        adresse_nom_voie            VARCHAR(200),
        code_postal                 VARCHAR(10),
        nom_commune                 VARCHAR(100),
        code_departement            VARCHAR(5),
        code_commune                VARCHAR(10),
        type_local                  VARCHAR(50),
        surface_reelle_bati         NUMERIC(10, 2),
        nombre_pieces_principales   INTEGER,
        longitude                   NUMERIC(10, 6),
        latitude                    NUMERIC(10, 6),
        prix_m2                     NUMERIC(10, 2),
        annee_mutation              INTEGER,
        mois_mutation               INTEGER,
        created_at                  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

    -- Index pour accélérer les requêtes fréquentes
    CREATE INDEX IF NOT EXISTS idx_dvf_raw_commune ON dvf_raw(nom_commune);
    CREATE INDEX IF NOT EXISTS idx_dvf_raw_type_local ON dvf_raw(type_local);
    CREATE INDEX IF NOT EXISTS idx_dvf_raw_date ON dvf_raw(date_mutation);
    CREATE INDEX IF NOT EXISTS idx_dvf_raw_code_postal ON dvf_raw(code_postal);

    -- ----------------------------------------------------------------
    -- Table 2 : Prix agrégés par arrondissement (zone curated)
    -- ----------------------------------------------------------------
    CREATE TABLE IF NOT EXISTS prix_m2_arrondissement (
        id                  SERIAL PRIMARY KEY,
        code_postal         VARCHAR(10) NOT NULL,
        arrondissement      INTEGER NOT NULL,
        annee               INTEGER NOT NULL,
        mois                INTEGER NOT NULL,
        prix_m2_moyen       NUMERIC(10, 2),
        prix_m2_median      NUMERIC(10, 2),
        prix_m2_min         NUMERIC(10, 2),
```

```
    prix_m2_max         NUMERIC(10, 2),
    nb_transactions     INTEGER,
    surface_moyenne     NUMERIC(10, 2),
    updated_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (code_postal, annee, mois)
);

CREATE INDEX IF NOT EXISTS idx_prix_arrdt ON prix_m2_arrondissement(arrondissement
CREATE INDEX IF NOT EXISTS idx_prix_annee_mois ON prix_m2_arrondissement(annee, mo

-- ----------------------------------------------------------------
-- Table 3 : Statistiques globales du marché parisien
-- ----------------------------------------------------------------
CREATE TABLE IF NOT EXISTS stats_marche (
    id                      SERIAL PRIMARY KEY,
    annee                   INTEGER NOT NULL,
    mois                    INTEGER NOT NULL,
    nb_transactions_total   INTEGER,
    prix_m2_median_paris    NUMERIC(10, 2),
    prix_m2_moyen_paris     NUMERIC(10, 2),
    arrdt_plus_cher         INTEGER,
    arrdt_moins_cher        INTEGER,
    surface_mediane         NUMERIC(10, 2),
    date_calcul             TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (annee, mois)
);
```

Vérification du schéma

Une fois la stack démarrée, vérifiez que les tables ont bien été créées :

```
$ Bash

docker exec -it dvf-postgres psql -U airflow -d dvf -c "\dt"
```

Partie 4 — Développement du DAG Airflow

Structure du fichier

Créez le fichier `dags/dag_dvf.py`. Le DAG doit être configuré comme suit :

Schéma d'architecture

```
"""
    DAG : Pipeline DVF — Data Lake (HDFS) vers Data Warehouse (PostgreSQL)
    Source : Demandes de Valeurs Foncières — data.gouv.fr
    """

    from __future__ import annotations

    import logging
    import os
    import tempfile
    from datetime import datetime, timedelta

    import pandas as pd
    import requests
    from airflow.decorators import dag, task
    from airflow.models import Variable
    from airflow.providers.postgres.hooks.postgres import PostgresHook
    from airflow.providers.postgres.operators.postgres import PostgresOperator
    from airflow.models.baseoperator import chain
    from airflow.utils.dates import days_ago

    logger = logging.getLogger(__name__)

    # ── Constantes ──────────────────────────────────────────────────────────

    DVF_URL = "https://www.data.gouv.fr/fr/datasets/r/90a98de0-f562-4328-aa16-fe0dd1dc
    WEBHDFS_BASE_URL = "http://hdfs-namenode:9870/webhdfs/v1"
    WEBHDFS_USER = "root"
    HDFS_RAW_PATH = "/data/dvf/raw"
    POSTGRES_CONN_ID = "dvf_postgres"

    # ── Configuration du DAG ────────────────────────────────────────────────

    default_args = {
        "owner": "data-engineering",
        "depends_on_past": False,
        "email_on_failure": False,
        "retries": 2,
        "retry_delay": timedelta(minutes=5),
    }


    @dag(
        dag_id="pipeline_dvf_immobilier",
        description="ETL DVF : téléchargement -> HDFS raw -> PostgreSQL curated",
        schedule_interval="0 6 * * *",   # tous les jours à 6h
        start_date=days_ago(1),
        catchup=False,
```

```
    default_args=default_args,
    tags=["dvf", "immobilier", "etl", "hdfs", "postgresql"],
)
def pipeline_dvf():
    # Les tâches seront définies ici (voir ci-dessous)
    pass


pipeline_dvf()
```

Tâche 1 — `verifier_sources()`

Rôle : Vérifier que l'API data.gouv.fr est accessible et que le cluster HDFS répond avant de lancer le pipeline. Retourne un dictionnaire avec le statut de chaque source.

Schéma d'architecture

```
@task(task_id="verifier_sources")
    def verifier_sources() -> dict:
        """
        Vérifie la disponibilité des sources de données et de l'infrastructure.

        Retourne un dictionnaire de la forme :
        {
            "dvf_api": True,
            "hdfs": True,
            "timestamp": "2024-01-15T06:00:00"
        }

        Lève une AirflowException si une source critique est indisponible.
        """
        statuts = {}

        # TODO 1 : Vérifier l'API data.gouv.fr
        # Envoyer une requête HEAD (ou GET avec stream=True) sur DVF_URL
        # Mettre un timeout court (ex: 10 secondes)
        # Stocker True/False dans statuts["dvf_api"]

        # TODO 2 : Vérifier la disponibilité de HDFS (WebHDFS)
        # Appeler l'endpoint : GET /webhdfs/v1/?op=LISTSTATUS
        # Stocker True/False dans statuts["hdfs"]

        # TODO 3 : Logger l'état de chaque service
        # Utiliser logger.info() pour chaque statut

        # TODO 4 : Lever une exception si HDFS ou l'API DVF est inaccessible

        statuts["timestamp"] = datetime.now().isoformat()
        return statuts
```

Tâche 2 — `telecharger_dvf()`

Rôle : Télécharger le fichier CSV DVF depuis data.gouv.fr et le sauvegarder temporairement sur le disque local du worker Airflow.

Schéma d'architecture

```
@task(task_id="telecharger_dvf")
    def telecharger_dvf(statuts: dict) -> str:
        """
        Télécharge le fichier CSV DVF depuis data.gouv.fr.

        Paramètre :
            statuts (dict) : résultat de verifier_sources()

        Retourne :
            str : chemin local du fichier CSV téléchargé (ex: /tmp/dvf_2023.csv)

        Le téléchargement doit être fait en streaming pour gérer les gros fichiers
        (le fichier DVF peut dépasser 500 Mo).
        """
        # TODO 1 : Construire le nom du fichier local
        # Utiliser tempfile.gettempdir() pour le répertoire temporaire
        # Nommer le fichier dvf_{YYYY}.csv avec l'année courante

        # TODO 2 : Télécharger en streaming
        # Utiliser requests.get(DVF_URL, stream=True, timeout=300)
        # Écrire par chunks de 8192 octets dans le fichier local
        # Afficher la progression (logger.info tous les 50 Mo)

        # TODO 3 : Vérifier que le fichier téléchargé n'est pas vide
        # Utiliser os.path.getsize()
        # Lever une exception si taille < 1000 octets

        # TODO 4 : Logger la taille finale du fichier et retourner le chemin

        local_path = None  # A remplacer
        return local_path
```

Tâche 3 — `stocker_hdfs_raw()`

Rôle : Uploader le fichier CSV dans la zone raw d'HDFS via l'API WebHDFS.

Schéma d'architecture

```
@task(task_id="stocker_hdfs_raw")
    def stocker_hdfs_raw(local_path: str) -> str:
        """
        Uploade le fichier CSV local vers HDFS (zone raw / data lake).

        Paramètre :
            local_path (str) : chemin du fichier CSV local (depuis telecharger_dvf)

        Retourne :
            str : chemin HDFS du fichier uploadé
                  (ex: /data/dvf/raw/dvf_2023.csv)

        Rappel de l'upload WebHDFS en deux étapes :
          Etape 1 - Initier : PUT NameNode -> 307 Redirect
          Etape 2 - Uploader : PUT DataNode (URL de redirection) avec les données
        """
        import os

        annee = datetime.now().year
        hdfs_filename = f"dvf_{annee}.csv"
        hdfs_file_path = f"{HDFS_RAW_PATH}/{hdfs_filename}"

        # TODO 1 : Créer le répertoire HDFS si nécessaire
        # URL : PUT {WEBHDFS_BASE_URL}{HDFS_RAW_PATH}/?op=MKDIRS&user.name={WEBHDFS_US
        # Vérifier que la réponse contient {"boolean": true}

        # TODO 2 : Initier l'upload (étape 1)
        # URL : PUT {WEBHDFS_BASE_URL}{hdfs_file_path}?op=CREATE&user.name=...&overwri
        # Utiliser allow_redirects=False
        # Récupérer l'URL de redirection dans response.headers["Location"]

        # TODO 3 : Envoyer le fichier vers le DataNode (étape 2)
        # Ouvrir le fichier local en mode "rb"
        # PUT sur l'URL de redirection avec le contenu binaire
        # Vérifier que le code HTTP est 201 Created

        # TODO 4 : Nettoyer le fichier temporaire local après upload
        # os.remove(local_path)

        # TODO 5 : Logger le chemin HDFS et retourner hdfs_file_path

        return hdfs_file_path
```

Tâche 4 — `traiter_donnees()`

Rôle : Lire le CSV depuis HDFS, filtrer les appartements parisiens, calculer le prix au m² et agréger par arrondissement.

Schéma d'architecture

```
@task(task_id="traiter_donnees")
    def traiter_donnees(hdfs_path: str) -> dict:
        """
        Lit le CSV depuis HDFS, applique les filtres métier et calcule les agrégats.

        Filtres à appliquer :
          - Type de bien : "Appartement" uniquement
          - Commune : Paris (codes postaux 75001 à 75020)
          - Surface : entre 9 m² et 500 m² (exclure les valeurs aberrantes)
          - Prix : supérieur à 10 000 EUR (exclure les valeurs aberrantes)
          - Nature de la mutation : "Vente"

        Agrégats à calculer par arrondissement (code postal) :
          - Prix moyen au m²
          - Prix médian au m²
          - Prix min et max au m²
          - Nombre de transactions
          - Surface moyenne

        Retourne :
            dict avec deux clés :
              "agregats" : liste de dicts (une ligne par arrondissement)
              "stats_globales" : dict avec les statistiques globales Paris
        """
        import io

        # TODO 1 : Lire le CSV depuis HDFS
        # GET {WEBHDFS_BASE_URL}{hdfs_path}?op=OPEN&user.name={WEBHDFS_USER}
        # Attention : WebHDFS redirige vers le DataNode (allow_redirects=True)
        # Lire le contenu bytes et le convertir en DataFrame pandas

        # TODO 2 : Renommer les colonnes si nécessaire
        # Les colonnes DVF incluent : "Date mutation", "Nature mutation",
        # "Valeur fonciere", "Code postal", "Type local",
        # "Surface reelle bati", "Nombre pieces principales"
        # Nettoyer les noms (minuscules, remplacer espaces par _)

        # TODO 3 : Filtrer les données
        # Appliquer les 5 filtres listés ci-dessus
        # Logger le nombre de lignes avant et après filtrage

        # TODO 4 : Calculer le prix au m²
        # prix_m2 = valeur_fonciere / surface_reelle_bati
        # Exclure les lignes où surface = 0

        # TODO 5 : Extraire l'arrondissement depuis le code postal
        # Ex : "75008" -> arrondissement 8
        # Ex : "75116" -> arrondissement 16
```

```
    # TODO 6 : Agréger par arrondissement
    # Utiliser groupby().agg() avec les agrégats listés ci-dessus

    # TODO 7 : Calculer les statistiques globales Paris

    # TODO 8 : Retourner le dictionnaire résultat

    return {"agregats": [], "stats_globales": {}}
```

Tâche 5 — `inserer_postgresql()`

Rôle : Insérer ou mettre à jour les agrégats dans PostgreSQL en utilisant `PostgresHook`.

Schéma d'architecture

```python
@task(task_id="inserer_postgresql")
    def inserer_postgresql(resultats: dict) -> int:
        """
        Insère les données agrégées dans PostgreSQL (zone curated).
        Utilise un UPSERT (INSERT ... ON CONFLICT DO UPDATE) pour l'idempotence.

        Paramètre :
            resultats (dict) : résultat de traiter_donnees()

        Retourne :
            int : nombre de lignes insérées ou mises à jour
        """
        hook = PostgresHook(postgres_conn_id=POSTGRES_CONN_ID)

        # TODO 1 : Récupérer les agrégats depuis le dictionnaire résultats
        agregats = resultats.get("agregats", [])
        stats_globales = resultats.get("stats_globales", {})

        # TODO 2 : Construire la requête UPSERT pour prix_m2_arrondissement
        # Utiliser INSERT INTO ... ON CONFLICT (code_postal, annee, mois)
        # DO UPDATE SET prix_m2_moyen = EXCLUDED.prix_m2_moyen, ...
        upsert_query = """
            INSERT INTO prix_m2_arrondissement
                (code_postal, arrondissement, annee, mois,
                 prix_m2_moyen, prix_m2_median, prix_m2_min, prix_m2_max,
                 nb_transactions, surface_moyenne, updated_at)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, NOW())
            ON CONFLICT (code_postal, annee, mois) DO UPDATE SET
                prix_m2_moyen    = EXCLUDED.prix_m2_moyen,
                prix_m2_median   = EXCLUDED.prix_m2_median,
                prix_m2_min      = EXCLUDED.prix_m2_min,
                prix_m2_max      = EXCLUDED.prix_m2_max,
                nb_transactions  = EXCLUDED.nb_transactions,
                surface_moyenne  = EXCLUDED.surface_moyenne,
                updated_at       = NOW();
        """

        # TODO 3 : Exécuter l'UPSERT pour chaque arrondissement
        # Utiliser hook.run() avec les paramètres appropriés
        # Attention : convertir les types numpy en types Python natifs (float(), int()

        # TODO 4 : Insérer les statistiques globales dans stats_marche
        # Même approche UPSERT sur (annee, mois)

        # TODO 5 : Retourner le nombre de lignes traitées

        return 0
```

Tâche 6 — `generer_rapport()`

Rôle : Interroger PostgreSQL pour afficher le classement des arrondissements parisiens par prix au m².

Schéma d'architecture

```
@task(task_id="generer_rapport")
    def generer_rapport(nb_inseres: int) -> str:
        """
        Génère un rapport SQL listant les arrondissements parisiens
        classés par prix médian au m² décroissant.

        Paramètre :
            nb_inseres (int) : nombre de lignes insérées (depuis inserer_postgresql)

        Retourne :
            str : rapport formaté (tableau texte)
        """
        hook = PostgresHook(postgres_conn_id=POSTGRES_CONN_ID)

        # TODO 1 : Écrire la requête SQL de ranking
        # Sélectionner : arrondissement, prix_m2_median, prix_m2_moyen,
        #                nb_transactions, surface_moyenne
        # Filtrer sur l'année et le mois courants
        # Trier par prix_m2_median DESC
        # Limiter à 20 résultats (les 20 arrondissements de Paris)
        query = """
            SELECT
                arrondissement,
                prix_m2_median,
                prix_m2_moyen,
                nb_transactions,
                surface_moyenne
            FROM prix_m2_arrondissement
            WHERE annee = %s AND mois = %s
            ORDER BY prix_m2_median DESC
            LIMIT 20;
        """

        # TODO 2 : Exécuter la requête avec hook.get_records()
        # Passer l'année et le mois courants en paramètres

        # TODO 3 : Formater les résultats en tableau lisible
        # Exemple de format attendu dans les logs :
        #
        # Arrondissement | Median (EUR/m2) | Moyen (EUR/m2) | Transactions
        # ---------------|-----------------|----------------|-------------
        # 6e             | 16 500          | 16 800         | 142
        # 4e             | 14 200          | 14 600         | 89
        # 7e             | 13 800          | 14 100         | 67

        # TODO 4 : Logger le rapport avec logger.info()
        # TODO 5 : Retourner le rapport sous forme de chaîne de caractères
```

```
        return ""
```

Enchaînement des tâches

A la fin de la fonction `pipeline_dvf()`, définissez l'ordre d'exécution avec `chain()` :

📐 **Schéma d'architecture**

```
# Instancier les tâches (à placer dans le corps de la fonction pipeline_dvf)
    t_verif    = verifier_sources()
    t_download = telecharger_dvf(t_verif)
    t_hdfs     = stocker_hdfs_raw(t_download)
    t_traiter  = traiter_donnees(t_hdfs)
    t_pg       = inserer_postgresql(t_traiter)
    t_rapport  = generer_rapport(t_pg)

    # Définir la chaîne de dépendances explicitement
    chain(t_verif, t_download, t_hdfs, t_traiter, t_pg, t_rapport)
```

Note : Avec la TaskFlow API (`@task`), les dépendances sont en grande partie inférées automatiquement par le passage des valeurs de retour en paramètre. L'appel à `chain()` est une bonne pratique pour les rendre explicites et lisibles.

Partie 5 — Vérification et Tests

5.1 Vérifier HDFS

Ouvrez l'interface web du NameNode à l'adresse http://localhost:9870.

Naviguez vers **Utilities > Browse the file system** et vérifiez la présence de `/data/dvf/raw/dvf_2023.csv`.

Vous pouvez également tester l'API WebHDFS depuis votre terminal :

Schéma d'architecture

```
# Lister le contenu du répertoire raw
    curl "http://localhost:9870/webhdfs/v1/data/dvf/raw/?op=LISTSTATUS&user.name=root"

    # Vérifier les métadonnées du fichier uploadé
    curl "http://localhost:9870/webhdfs/v1/data/dvf/raw/dvf_2023.csv?op=GETFILESTATUS&
```

5.2 Vérifier PostgreSQL

Schéma d'architecture

```
# Connexion à la base dvf
    docker exec -it dvf-postgres psql -U airflow -d dvf
```

Requêtes de vérification à exécuter dans psql :

Schéma d'architecture

```
-- Nombre total d'enregistrements
    SELECT COUNT(*) FROM dvf_raw;
    SELECT COUNT(*) FROM prix_m2_arrondissement;

    -- Afficher le top 5 des arrondissements les plus chers
    SELECT
        arrondissement,
        ROUND(prix_m2_median, 0) AS median_m2_eur,
        nb_transactions
    FROM prix_m2_arrondissement
    ORDER BY prix_m2_median DESC
    LIMIT 5;

    -- Vérifier les statistiques globales
    SELECT * FROM stats_marche ORDER BY date_calcul DESC LIMIT 1;
```

5.3 Déclencher et surveiller le DAG

Schéma d'architecture

```
# Déclencher manuellement le DAG depuis la CLI Airflow
    docker exec -it dvf-airflow-scheduler airflow dags trigger pipeline_dvf_immobilier

    # Lister les derniers runs du DAG
    docker exec -it dvf-airflow-scheduler airflow dags list-runs -d pipeline_dvf_immob
```

Depuis l'interface web Airflow (http://localhost:8080) :

1. Activez le DAG `pipeline_dvf_immobilier` (toggle en haut à gauche de la liste)
2. Cliquez sur **Trigger DAG** (bouton Play)
3. Ouvrez la vue **Graph** pour suivre la progression tâche par tâche
4. Cliquez sur chaque tâche pour consulter les logs d'exécution

5.4 Checklist de validation

- ☐ La stack Docker démarre sans erreur (`docker compose ps` montre tous les services `healthy`)
- ☐ L'interface HDFS NameNode est accessible sur http://localhost:9870
- ☐ L'interface Airflow est accessible sur http://localhost:8080
- ☐ Le DAG `pipeline_dvf_immobilier` apparaît sans erreur d'import
- ☐ La tâche `verifier_sources` passe en vert (sources accessibles)
- ☐ La tâche `telecharger_dvf` télécharge un fichier de taille > 1 Mo
- ☐ La tâche `stocker_hdfs_raw` crée le fichier dans HDFS (visible dans le browser HDFS)
- ☐ La tâche `traiter_donnees` affiche dans les logs le nombre de lignes filtrées
- ☐ La tâche `inserer_postgresql` insère des données dans `prix_m2_arrondissement`
- ☐ La tâche `generer_rapport` affiche un classement des arrondissements dans les logs

---

Critères d'évaluation

Critère	Points
Infrastructure Docker	
`docker-compose.yaml` complet et fonctionnel	3 pts

Critère	Points
HDFS (NameNode + DataNode) accessible	2 pts
PostgreSQL avec 2 bases (airflow + dvf)	1 pt
Schéma PostgreSQL	
Script `init_dvf.sql` correct et complet	2 pts
Index pertinents créés	1 pt
DAG Airflow	
Structure du DAG et configuration (schedule, tags)	2 pts
`verifier_sources` fonctionnel	2 pts
`telecharger_dvf` avec streaming correct	3 pts
`stocker_hdfs_raw` (double PUT WebHDFS)	4 pts
`traiter_donnees` : filtrage + agrégation	4 pts
`inserer_postgresql` avec UPSERT idempotent	3 pts
`generer_rapport` avec classement formaté	2 pts
Enchaînement des tâches avec `chain()`	1 pt
Qualité du code	
Gestion des erreurs (try/except, logs)	2 pts
Code lisible, commenté, PEP8	1 pt
XCom correctement utilisés (valeurs de retour)	1 pt
Total	**34 pts**

Bonus

Bonus 1 — Partitionnement HDFS par année et département (+ 3 pts)

Modifiez `stocker_hdfs_raw` pour organiser les fichiers dans une structure partitionnée :

```
/data/dvf/raw/
    annee=2023/
        dept=75/
            dvf_2023_75.csv
        dept=92/
            dvf_2023_92.csv
    annee=2024/
        ...
```

Cette structure respecte la convention **Hive-style partitioning** compatible avec Apache Spark et Apache Hive. Elle permet de lire uniquement les partitions nécessaires lors du traitement (partition pruning).

Bonus 2 — Analyse de tendance mois sur mois (+ 3 pts)

Ajoutez une tâche `analyser_tendances()` après `generer_rapport()` qui calcule l'évolution du prix au m² entre le mois courant et le mois précédent. La requête SQL de base à compléter :

```SQL
SELECT
        a.arrondissement,
        a.prix_m2_median                                                    AS prix
        b.prix_m2_median                                                    AS prix
        ROUND(
            ((a.prix_m2_median - b.prix_m2_median) / b.prix_m2_median) * 100,
            2
        )                                                                   AS vari
    FROM prix_m2_arrondissement a
    JOIN prix_m2_arrondissement b
        ON  a.arrondissement = b.arrondissement
        AND a.annee          = b.annee
        AND a.mois           = b.mois + 1
    WHERE a.annee = EXTRACT(YEAR FROM NOW())
    ORDER BY variation_pct DESC;
```

Ajoutez cette tâche à la chaîne de dépendances et stockez le résultat dans la table `stats_marche`.

Bonus 3 — Second DataNode (+ 2 pts)

Ajoutez un second service `hdfs-datanode-2` dans le `docker-compose.yaml` pour simuler un cluster HDFS avec réplication des blocs. Configurez le facteur de réplication à 2 via la variable d'environnement `HDFS_CONF_dfs_replication=2`. Vérifiez dans l'interface NameNode que :

- Le cluster indique **2 DataNodes actifs** (section Datanodes)
- Les blocs du fichier DVF sont répliqués sur les deux noeuds (section Block Pool)

Ressources

Ressource	URL
DVF open data — data.gouv.fr	https://www.data.gouv.fr/fr/datasets/demandes-de-valeurs-foncieres/
Fichier CSV DVF 2023 (téléchargement direct)	https://www.data.gouv.fr/fr/datasets/r/90a98de0-f562-4328-aa16-fe0dd1dca60f
WebHDFS REST API (doc officielle)	https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/WebHDFS.html
TaskFlow API Airflow	https://airflow.apache.org/docs/apache-airflow/stable/tutorial/taskflow.html
PostgresHook — doc Airflow	https://airflow.apache.org/docs/apache-airflow-providers-postgres/stable/index.html
Image Docker Hadoop (bde2020)	https://github.com/big-data-europe/docker-hadoop
Documentation DVF — DGFiP	https://www.data.gouv.fr/fr/datasets/demandes-de-valeurs-foncieres/#description

---

TP rédigé pour la formation Apache Airflow — IPSSI Montpellier — Jour 3

Exercices Supplémentaires

Ces exercices approfondissent la qualité des données, le partitionnement HDFS et l'optimisation PostgreSQL.

Exercice 1 — Contrôle qualité des données DVF (⭐⭐)

Contexte : Les données DVF contiennent des valeurs aberrantes (prix à 1€, surfaces à 0 m², transactions en doublon). En production, un pipeline sans contrôle qualité génère des rapports erronés.

Objectif : Insérer une tâche `controler_qualite(transactions)` entre `traiter_dvf` et `calculer_stats`.

RÈGLES DE QUALITÉ À IMPLÉMENTER

Schéma d'architecture

```
@task()
    def controler_qualite(transactions: list[dict]) -> dict:
        """
        Applique les règles de qualité OMS-DGFiP sur les transactions DVF.
        Retourne un rapport qualité + la liste filtrée (transactions valides uniquemen
        """
        rapport = {
            "nb_total":              len(transactions),
            "nb_prix_aberrant":      0,   # valeur_fonciere < 1000 € ou > 50 000 000 €
            "nb_surface_aberrante":  0,   # surface < 5 m² ou > 10 000 m²
            "nb_prix_m2_aberrant":   0,   # prix/m² < 100 €/m² ou > 50 000 €/m²
            "nb_doublons":          0,    # même (date, commune, surface, valeur)
            "nb_valides":           0,
            "taux_validite_pct":    0.0,
        }

        # TODO : Implémenter chaque règle
        # Règle 1 — Prix aberrant
        # Règle 2 — Surface aberrante
        # Règle 3 — Prix/m² aberrant (calculer prix_m2 = valeur / surface)
        # Règle 4 — Doublons : utiliser un set de tuples (date, commune, surface, vale

        # Si moins de 70% des transactions sont valides → lever une ValueError
        if rapport["taux_validite_pct"] < 70.0:
            raise ValueError(
                f"Qualité insuffisante : {rapport['taux_validite_pct']:.1f}% valide "
                f"(seuil 70%) — pipeline interrompu"
            )

        log.info(
            "Qualité DVF : %d/%d valides (%.1f%%) | Prix aberrants: %d | "
            "Surfaces aberrantes: %d | Doublons: %d",
            rapport["nb_valides"], rapport["nb_total"],
            rapport["taux_validite_pct"],
            rapport["nb_prix_aberrant"], rapport["nb_surface_aberrante"], rapport["nb_
        )

        return rapport   # Le XCom contient le rapport + la liste filtrée (à adapter)
```

TRAVAIL DEMANDÉ

1. Implémentez toutes les règles de qualité.
2. Ajoutez la tâche dans le pipeline : `traiter_dvf → controler_qualite → calculer_stats`.
3. Persistez le rapport qualité dans une table PostgreSQL `dvf_qualite_runs` :

```sql
CREATE TABLE IF NOT EXISTS dvf_qualite_runs (
        id                SERIAL PRIMARY KEY,
        date_run          TIMESTAMP DEFAULT NOW(),
        nb_total          INTEGER,
        nb_valides        INTEGER,
        taux_validite     FLOAT,
        nb_prix_aberrant  INTEGER,
        nb_doublons       INTEGER
    );
```

4. Testez avec des données intentionnellement corrompues (modifiez quelques lignes du CSV).

Question : Quelle `TriggerRule` utiliser sur `generer_rapport` si vous voulez qu'il s'exécute même quand `controler_qualite` lève une ValueError ?

Exercice 2 — Partitionnement HDFS par année et département (⭐⭐⭐)

Contexte : Stocker un seul fichier DVF de 200 Mo dans HDFS n'est pas optimal. En production, les données sont **partitionnées** pour accélérer les lectures (Hive/Spark ne lisent que les partitions nécessaires).

Objectif : Modifier `stocker_hdfs_raw` pour partitionner les données DVF.

STRUCTURE CIBLE DANS HDFS

Schéma d'architecture

```
/dvf/
    ├── raw/
    │   ├── annee=2023/
    │   │   ├── dept=75/dvf_75_2023.csv   ← Paris
    │   │   ├── dept=69/dvf_69_2023.csv   ← Rhône
    │   │   └── dept=13/dvf_13_2023.csv   ← Bouches-du-Rhône
    │   └── annee=2024/
    │       └── dept=75/dvf_75_2024.csv
    └── processed/
        └── stats_commune_2023.json
```

TRAVAIL DEMANDÉ

1. Modifiez `traiter_dvf()` pour **grouper** les transactions par `(code_departement, annee)`. Le code département se dérive du code postal : `dept = str(code_postal)[:2]`.

2. Modifiez `stocker_hdfs_raw()` pour **uploader un fichier par partition** :

```python
for (dept, annee), transactions in groupes.items():
        hdfs_path = f"/dvf/raw/annee={annee}/dept={dept}/dvf_{dept}_{annee}.csv"
        # Upload via WebHDFS
```

3. Vérifiez dans l'UI HDFS (`http://localhost:9870`) que l'arborescence est correcte.

4. Adaptez `calculer_stats()` pour lire depuis la partition `annee=2023/dept=75/` uniquement.

5. **Benchmark** : Mesurez le temps de lecture d'une partition vs le fichier complet.

Question : Pourquoi cette structure est-elle compatible avec Apache Spark et Hive ? Quel format de fichier serait encore plus optimal que CSV pour HDFS ? (Parquet, ORC, Avro — expliquez le choix)

Exercice 3 — Vue matérialisée PostgreSQL et rapport d'évolution mensuelle (⭐⭐)

Contexte : Le responsable produit veut un graphe montrant l'évolution des prix au m² mois par mois sur 2 ans. Une requête SQL directe sur 200k transactions prendrait trop de temps.

Objectif : Créer une **Materialized View** PostgreSQL et une tâche Airflow pour la rafraîchir.

CONCEPT MATERIALIZED VIEW

Schéma d'architecture

```
-- Vue matérialisée : résultat pré-calculé, stocké physiquement
    CREATE MATERIALIZED VIEW dvf_evolution_mensuelle AS
    SELECT
        code_postal,
        commune,
        EXTRACT(YEAR  FROM date_mutation) AS annee,
        EXTRACT(MONTH FROM date_mutation) AS mois,
        COUNT(*)                          AS nb_transactions,
        ROUND(AVG(valeur_fonciere / NULLIF(surface_reelle_bati, 0))::numeric, 0) AS pr
        PERCENTILE_CONT(0.5) WITHIN GROUP (
            ORDER BY valeur_fonciere / NULLIF(surface_reelle_bati, 0)
        )                                 AS prix_median_m2
    FROM dvf_transactions
    WHERE surface_reelle_bati > 0
      AND valeur_fonciere > 0
    GROUP BY code_postal, commune,
             EXTRACT(YEAR FROM date_mutation),
             EXTRACT(MONTH FROM date_mutation)
    WITH DATA;

    -- Index pour accélérer les requêtes
    CREATE INDEX ON dvf_evolution_mensuelle (code_postal, annee, mois);
```

TRAVAIL DEMANDÉ

1. Ajoutez la création de la Materialized View dans `SQL_CREER_TABLES` (avec `IF NOT EXISTS` — astuce : `CREATE MATERIALIZED VIEW IF NOT EXISTS` n'existe pas, utilisez `DO $$ ... $$`).
2. Créez une tâche Airflow `rafraichir_vue_materialisee()` :

📐 **Schéma d'architecture**

```
@task()
    def rafraichir_vue_materialisee() -> int:
        """
        Rafraîchit la Materialized View après chaque ingestion.
        CONCURRENTLY : rafraîchissement sans bloquer les lectures.
        Nécessite un UNIQUE INDEX sur la vue.
        """
        hook = PostgresHook(postgres_conn_id=PG_CONN_ID)
        with hook.get_conn() as conn, conn.cursor() as cur:
            cur.execute("REFRESH MATERIALIZED VIEW CONCURRENTLY dvf_evolution_mensu
            conn.commit()
        # Retourner le nombre de lignes dans la vue
        count = hook.get_first("SELECT COUNT(*) FROM dvf_evolution_mensuelle")[0]
        log.info("Vue matérialisée rafraîchie : %d lignes", count)
        return count
```

3. Ajoutez cette tâche après `inserer_postgresql` dans le pipeline.

4. Écrivez une requête SQL de rapport d'évolution pour Paris (75) :

🗄 SQL

```sql
SELECT annee, mois, nb_transactions, prix_moyen_m2,
           prix_moyen_m2 - LAG(prix_moyen_m2) OVER (ORDER BY annee, mois) AS variati
    FROM dvf_evolution_mensuelle
    WHERE code_postal LIKE '75%'
    ORDER BY annee, mois;
```

5. Intégrez cette requête dans `generer_rapport()` et affichez la variation mensuelle dans les logs.

Question : Quelle est la différence entre `VIEW` et `MATERIALIZED VIEW` en termes de performance et de fraîcheur des données ? Dans quel contexte choisiriez-vous l'un plutôt que l'autre ?

Exercices supplémentaires — Jour 3 — Formation Airflow IPSSI Montpellier